Unidad 5: Pruebas y Manejo de Errores

Tema 5.1: Pruebas Unitarias y Automatizadas

Área de Ciencias de la Computación

Tecnologías Orientadas a Objetos

23 de octubre de 2026

Pruebas Unitarias

Definición Fundamental

Las pruebas unitarias son pruebas que verifican el funcionamiento correcto de pequeñas partes del programa, como métodos o funciones específicas.

Características principales:

- **Aislamiento:** Su objetivo es evaluar que cada componente funcione de manera independiente, sin depender de bases de datos, redes u otros módulos.
- **Detección temprana:** Permiten identificar errores rápidamente durante la fase de desarrollo, reduciendo costos y tiempo de depuración.
- **Documentación viva:** Actúan como un manual de uso técnico, demostrando cómo debe comportarse exactamente el código ante diversas entradas.

Automatización de Pruebas

Concepto

Consiste en ejecutar pruebas mediante herramientas de software especializadas sin necesidad de intervención manual repetitiva.

Beneficios estratégicos:

- **Eficiencia:** Ahorra tiempo crítico al ejecutar miles de validaciones en segundos.
- **Prevención de Regresiones:** Detecta errores constantemente durante el ciclo de desarrollo, asegurando que el código nuevo no rompa las funcionalidades previas.
- **Refactorización segura:** Otorga confianza al programador para optimizar y limpiar el código, respaldado por una red de seguridad automatizada.

El Patrón Given-When-Then (GWT)

Para estructurar pruebas legibles y mantenibles, se utiliza el patrón **Given-When-Then** (Dado-Cuando-Entonces), aislando claramente las fases de la prueba:

1. **Given (Dado - Preparación):** Establece el contexto inicial. Se instancian los objetos, se configuran las variables y se prepara el entorno necesario para la prueba.
2. **When (Cuando - Ejecución):** Es la invocación del método o la acción específica que se desea someter a prueba. Idealmente es una sola línea de código.
3. **Then (Entonces - Verificación):** Comprueba que el resultado obtenido coincide con el resultado esperado utilizando aserciones (*asserts*).

Aserciones (Asserts): Conceptos y Tipos

Los *Asserts* son métodos que comparan el resultado esperado con el resultado real. Si la condición no se cumple, la prueba falla (Rojo); si se cumple, pasa (Verde).

1. Igualdad y Diferencia

```
1 // Verifica si dos valores son iguales
2 assertEquals(resultadoEsperado, resultadoReal, "Mensaje
      opcional de fallo");
3
4 // Verifica si dos valores son diferentes
5 assertNotEquals(valorInesperado, resultadoReal);
6
```

2. Condiciones Booleanas

```
1 // Verifica que una condicion sea verdadera
2 assertTrue(condicion, "La condicion debia ser true");
3
4 // Verifica que una condicion sea falsa
5 assertFalse(condicion);
6
```

Aserciones (Asserts): Tipos Avanzados

3. Nulos y Referencias

```
// Verifica que un objeto sea nulo
assertNull(objeto, "El objeto debe ser null");

// Verifica que un objeto haya sido instanciado
assertNotNull(objeto);

```

4. Manejo de Excepciones Fundamental para la programación defensiva. Verifica que un método lance el error correcto ante datos inválidos.

```
assertThrows(IllegalArgumentException.class, () -> {
    // Codigo que debe detonar la excepcion
    objeto.metodoConDatosInvalidos(-1);
});

```

Implementación: Given, When, Then

```java
@Test
public void testCalcularDescuentoValido() {
    // 1. GIVEN (Configuracion del escenario)
    CalculadoraVentas calc = new CalculadoraVentas();
    double precioOriginal = 100.0;
    double porcentajeDescuento = 20.0;

    // 2. WHEN (Ejecucion del comportamiento)
    double precioFinal = calc.aplicarDescuento(
    precioOriginal, porcentajeDescuento);

    // 3. THEN (Verificacion del resultado)
    assertEquals(80.0, precioFinal, "El descuento del 20% no
     se aplico correctamente");
}

```

Listing 1: Ejemplo integrador en JUnit 5